

JAVA SDE-2 INTERVIEW PREPARATION SERIES

VOLUME 18 OF 18 - PART H OF J

Advanced Java H: Spring Boot and REST APIs

Dependency Injection, MVC Contracts, Transactions, Idempotency,
Operations, and Testing

FOUNDING AUTHOR

Vinay Reddy Kalluri

EDITOR-IN-CHIEF | CHIEF AUDITOR

Build Spring services from explicit object graphs and durable HTTP contracts, then harden transaction, idempotency, security, observability, and testing boundaries.

SPRING BOOT | DI | MVC | REST | TRANSACTIONS | TESTING

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Release 2026-07-25

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

60-second entry rule: If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to [18G – Advanced Java G – Interview Revision](#). If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Part H opens the backend specialist track with the Spring and HTTP reasoning expected in Java SDE-2 interviews, while keeping framework behavior tied to explicit contracts and failure modes.

Readiness map

Decision	Evidence
START HERE	A Spring proxy, scope, or transaction boundary changes behavior; An HTTP API needs durable status, error, pagination, or concurrency semantics; Retries require idempotency rather than duplicate side effects; A service must expose safe readiness, metrics, and tests.
PREPARE FIRST	Stage 18B language design and 18F backend boundaries; HTTP, SQL, and testing fundamentals.
MOVE ON WHEN	Explain Spring container, proxy, configuration, and transaction behavior; Design stable REST request, response, error, and pagination contracts; Implement idempotency and operational boundaries; Choose unit, MVC, integration, and end-to-end evidence.

Choose the route that fits today

- 1 **Learning:** read in order, type the representative Java examples, and complete each dry run.
- 2 **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Volume Position

18G - Advanced Java G - Interview Revision	YOU ARE HERE 18H - Advanced Java H - Spring	18I - Advanced Java I - Persistence
--	--	-------------------------------------

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION	
Start Here - Your Route Through This Volume	2
Part I - Core Learning	4
Chapter 1 - Spring Container, Configuration, Proxies, and Transaction Boundaries	4
Chapter 2 - Spring MVC and Durable REST Contracts	19
Chapter 3 - Idempotent APIs, Operational Boundaries, and Spring Testing	33
Chapter 4 - Worked Spring Service: A Retry-Safe Order API	43
Chapter 5 - Executable Spring Boundary Model	55
Part II - Practice and Handoff	64
Practice Ladder and Next Steps	64

Part I - Core Learning**SDE-2 CORE CHAPTER**

Chapter 1 - Spring Container, Configuration, Proxies, and Transaction Boundaries

Learning objectives

After this chapter, you should be able to:

- explain inversion of control as a graph-construction and lifecycle problem rather than as annotation magic;
- choose constructor injection, qualifiers, scopes, and lifecycle hooks from explicit ownership rules;
- read Spring Boot auto-configuration as conditional configuration that backs away when user configuration is present;
- predict when a Spring AOP proxy can and cannot intercept a call;
- place a transaction around a business invariant and recognize proxy, rollback, and propagation traps; and
- test container wiring without turning every unit test into a full application startup.

1. The container is a managed object graph

Intuition and formal model

In ordinary Java, one object constructs another:

JAVA

```
var repository = new JdbcOrderRepository(dataSource);  
var service = new OrderService(repository, clock);
```

Dependency injection changes *who assembles the graph*, not the semantics of the objects. A Spring `ApplicationContext` reads bean definitions, resolves dependencies, creates instances, applies container extension points, and owns destruction callbacks. Let the bean-definition graph be $G = (V, E)$, where each vertex is a bean definition and an edge $A \rightarrow B$ means A requires B during construction. Eager singleton creation needs an acyclic resolvable subgraph. A constructor cycle is a design signal because neither instance can be completed first.

Inversion of control is therefore an ownership contract:

- 1 application code declares required capabilities;
- 2 configuration maps each capability to a concrete bean;

- 3 the container constructs and decorates the graph;
- 4 callers use the resulting objects without locating dependencies globally.

This is different from the service-locator pattern. Calling `applicationContext.getBean(...)` throughout business code hides required dependencies and couples the domain to a container. Injection keeps requirements visible in constructors and makes plain-Java testing possible.

Recognition and decision rules

Situation	Preferred decision	Why
dependency is required for every valid instance	constructor parameter	construction proves completeness and supports <code>final</code> fields
dependency is one of several implementations	qualifier or explicit <code>@Bean</code> wiring	selection becomes configuration, not ordering accident
dependency is optional policy	inject an explicit no-op/default implementation	avoids nullable or container-aware domain logic
two services depend on each other	redesign the responsibility boundary	lazy injection hides the cycle but rarely fixes cohesion
third-party class must be managed	<code>@Bean</code> factory method	classpath scanning cannot annotate code you do not own
many fields are requested by one service	examine cohesion before adding injection tricks	a large constructor often reveals multiple responsibilities

Concrete Spring example - dependency-requiring

The following snippet requires Spring Framework. It deliberately injects a small interface rather than a concrete transport:

JAVA

```
public interface PaymentGateway {
    Authorization authorize(String accountId, long amountCents);
}

@Service
final class CheckoutService {
    private final PaymentGateway gateway;
    private final OrderRepository orders;
    private final Clock clock;

    public CheckoutService(
        @Qualifier("primaryGateway") PaymentGateway gateway,
        OrderRepository orders,
        Clock clock) {
        this.gateway = Objects.requireNonNull(gateway);
        this.orders = Objects.requireNonNull(orders);
        this.clock = Objects.requireNonNull(clock);
    }
}

@Configuration(proxyBeanMethods = false)
class CheckoutConfiguration {
    @Bean
    Clock applicationClock() {
        return Clock.systemUTC();
    }
}
```

The important guarantee is visible without Spring: a `CheckoutService` cannot exist without its collaborators. `@Qualifier` makes selection explicit. The meaning of `proxyBeanMethods` is framework-specific; do not infer container behavior from the Java syntax alone.

Execution walkthrough

For an eager singleton service, a simplified startup sequence is:

- 1 configuration and component scanning contribute bean definitions;
- 2 post-processors may transform those definitions;
- 3 the container resolves the `CheckoutService` constructor;
- 4 it creates or obtains `PaymentGateway`, `OrderRepository`, and `Clock` beans;
- 5 it invokes the constructor;
- 6 it performs documented initialization callbacks and bean post-processing;
- 7 a post-processor may return a proxy that wraps the target;
- 8 the context publishes the fully initialized bean reference;
- 9 at orderly shutdown, destruction callbacks run for beans whose lifecycle the context owns.

The exact internal call sequence is not an application guarantee. The engineering contract is that extension points and lifecycle callbacks follow Spring's documented ordering rules. If ordering matters, test it against the supported framework version.

2. Lifecycle, scopes, and safe state

Bean lifecycle without folklore

A useful lifecycle model is:

TEXT

```
definition -> dependency resolution -> construction -> population
           -> aware callbacks / post-processors -> initialization
           -> ready reference (possibly proxy) -> destruction
```

Prefer constructors for establishing invariants. Use `@PostConstruct` only for initialization that needs injected collaborators and cannot be done in a constructor. It must not start unbounded background work that the application cannot stop. Pair acquired resources with `@PreDestroy` or another explicit lifecycle mechanism. A factory method that returns an object does not automatically give the container knowledge of every resource the object creates later.

Spring's default application scope is singleton *per container and bean definition*, not one instance per JVM and not automatically thread-safe. A singleton controller can serve concurrent requests. Keep it immutable or protect mutable state with a deliberate concurrency design. Do not store request-specific data in fields.

Common scopes have different ownership:

Scope	Identity and lifetime	Typical use	Primary risk
singleton	one managed instance per definition in a context	stateless service, repository, configuration	shared mutable state and unsafe publication assumptions
prototype	new instance each time the container supplies it	uncommon stateful helper	container does not manage the full destruction lifecycle in the same way as singletons
request	one instance for an HTTP request	request metadata or aggregation	access outside an active request; accidental capture by long-lived tasks
session	one instance per HTTP session	limited user-session state	memory growth, clustering/serialization, concurrent requests in one session
application	servlet-context lifetime	servlet application state	confusion with Spring singleton boundaries

Injecting a shorter-lived bean directly into a singleton needs an indirection such as a scoped proxy or provider. The decision rule is ownership: the long-lived object must not capture one short-lived instance forever. Prefer passing request data as method arguments when that is clearer.

Failure walkthrough: mutable singleton controller

JAVA

```
@RestController
final class UnsafeSequenceController {
    private long next = 1; // shared by concurrent requests

    @PostMapping("/numbers")
    long allocate() {
        return next++;
    }
}
```

Two request threads can read the same value and lose an increment. Even replacing `long` with `AtomicLong` would only make this local counter atomic; it would not make it durable, globally unique across instances, or transactionally connected to database work. The correct design begins with the required identity contract, often a database sequence or an external ID strategy.

3. Configuration and Boot auto-configuration

Configuration is precedence, conditions, and binding

Spring configuration combines bean definitions and an **Environment** of property sources. Spring Boot adds conventions and conditional auto-configuration. The correct mental model is not "Boot guesses." It is:

TEXT

```
explicit application configuration
+ discovered auto-configuration candidates
+ conditions (classes, beans, properties, web type, resources)
= effective bean graph
```

Auto-configuration commonly activates when required classes are present and expected user beans are absent. User configuration can make a default back away. Conditions and ordering belong to the supported Boot contract; the contents of auto-configuration classes are not an application API unless documentation says otherwise.

Use typed configuration properties for a cohesive group:

JAVA

```
// Dependency-requiring: Spring Boot configuration binding.
@ConfigurationProperties("payments")
public record PaymentProperties(
    URI baseUrl,
    Duration connectTimeout,
    Duration responseTimeout,
    int maxAttempts) {

    public PaymentProperties {
        Objects.requireNonNull(baseUrl);
        Objects.requireNonNull(connectTimeout);
        Objects.requireNonNull(responseTimeout);
        if (maxAttempts < 1 || maxAttempts > 5) {
            throw new IllegalArgumentException("maxAttempts must be 1..5");
        }
    }
}
```

Binding provides values; validation protects the contract. Secrets should enter through an approved secret mechanism, should never be rendered by an endpoint, and should not be copied into exception messages. Configuration changes that require restart must be treated differently from dynamic control-plane changes.

Diagnosing auto-configuration

When a bean is missing or duplicated, do not add annotations randomly. Inspect:

- 1 the dependency graph and active profiles;
- 2 the configuration-properties prefix and actual value source;
- 3 conditional evaluation or startup report;
- 4 bean names, types, qualifiers, and primary selection;
- 5 component-scan boundaries;
- 6 test-slice exclusions;
- 7 framework and BOM compatibility.

A condition report is evidence, not part of business logic. A robust application does not depend on auto-configuration implementation classes that official documentation labels internal.

4. Proxies and AOP: reason from the call path

Formal interception model

Many Spring services are exposed as proxies to apply transactions, method security, caching, metrics, or other advice. Model a proxied call as:

TEXT

```
caller -> proxy -> ordered advice chain -> target method
```

Advice can run only if the call passes through the relevant proxy. A call from one target method to another on **this** is an ordinary Java call and can bypass proxy advice. Private methods are not a reliable proxy interception boundary. Final classes/methods constrain subclass-proxy strategies. Interface-based and class-based proxies have different type surfaces. The exact strategy is configuration and framework-version sensitive.

Recognition rule: whenever an annotation appears to change method execution, draw the runtime reference and call path. Ask which object the caller holds. Never reason from the annotation alone.

Self-invocation failure

JAVA

```
@Service
class ImportService {
    void importBatch(List<Row> rows) {
        for (Row row : rows) {
            importOne(row); // direct call on this
        }
    }

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    void importOne(Row row) {
        // The expected separate transaction may not be created.
    }
}
```

The intended boundary is "one transaction per row," but the call path never re-enters the proxy. Better designs include moving the per-row use case to a separate injected bean, using an explicit transaction template, or changing the unit of work. Injecting "self" is usually a warning sign because it teaches application code to accommodate interception mechanics.

Advice ordering and failure semantics

If retry and transaction advice both apply, order changes behavior:

TEXT

```
retry outside transaction: each attempt may receive a fresh transaction
transaction outside retry: retries may occur inside one doomed transaction
```

The correct choice depends on the recoverable failure and transaction manager. Declare and test the intended ordering. Metrics advice must not accidentally transform exceptions. Security must occur before protected state changes. Caching around a mutating method requires explicit invalidation semantics.

5. Transaction boundaries and traps

The business invariant defines the boundary

Suppose confirming an order must transition **PENDING** → **CONFIRMED**, reserve inventory, and append an outbox event atomically in one database. That use case-not each repository call-is the natural local transaction boundary.

JAVA

```
// Dependency-requiring: Spring transaction support.
@Service
final class ConfirmOrderUseCase {
    private final OrderRepository orders;
    private final InventoryRepository inventory;
    private final OutboxRepository outbox;

    @Transactional
    public Confirmation confirm(OrderId id, long expectedVersion) {
        Order order = orders.require(id);
        order.confirm(expectedVersion);
        inventory.reserve(order.lines());
        outbox.append(OrderConfirmed.from(order));
        return Confirmation.from(order);
    }
}
```

The remote publication is not inside the database's atomic domain. An outbox relay publishes after commit and consumers deduplicate by event identity. Calling a broker or payment provider while holding locks expands latency and still cannot create atomicity across independent systems.

Rollback is policy, not punctuation

Declarative transactions decide rollback using the configured transaction manager and rollback rules. Do not memorize a one-line rule and assume every exception hierarchy matches it. Define domain failures deliberately, inspect the supported Spring behavior, and add integration tests that verify committed database state.

Important traps:

- catching an exception inside the method and returning success can allow commit;
- a transaction may already be marked rollback-only, causing failure at commit even after a catch;
- an asynchronous continuation normally runs in another thread and does not inherit a thread-bound transaction;
- `readOnly = true` is a hint/optimization contract, not a universal database authorization mechanism;
- a timeout is not proof that remote or database work did not occur;
- `REQUIRES_NEW` can consume another pooled connection while the outer transaction holds one;
- a long transaction increases lock duration, version conflict probability, and pool occupancy;
- an isolation name does not guarantee identical vendor behavior.

Propagation decision table

Need	Candidate	Reasoning and caveat
join an existing use-case transaction, otherwise create one	required/default behavior	common service boundary
run audit work independently of caller outcome	new transaction may be considered	can persist even when business work fails; connection pressure and semantic surprise
prohibit accidental transaction	never-style policy	useful for remote calls or orchestration that must not hold database resources
savepoint-like nested work	nested behavior if supported	manager/resource dependent; do not equate with an independent transaction
explicit complex control flow	transaction template/API	makes begin/commit scope visible and avoids self-invocation ambiguity

Propagation does not solve distributed transactions, idempotency, or event publication. It only specifies interaction with a transaction context understood by a manager.

6. Testing strategy

Use the smallest test that can falsify the claim:

- **plain unit test:** construct a service directly; test decisions and domain transitions;
- **container wiring test:** load a narrow configuration; assert bean selection and properties validation;
- **proxy behavior test:** use a real Spring context when interception is the subject;
- **database integration test:** use the production database engine/version when isolation, SQL, or locking matters;
- **end-to-end test:** verify request mapping, security, serialization, transaction, and persistent state for a few critical paths.

Mocking a repository cannot prove transaction rollback. Calling `new ConfirmOrderUseCase(...)` cannot prove proxy advice. Conversely, a full application test is a slow and opaque way to test a pure state transition.

TRY IT | 7. Interview questions and model checkpoints

Q1. Why is constructor injection preferred?

Model checkpoint: it exposes required dependencies, allows immutable fields, makes invalid partial construction difficult, and supports plain-Java tests. It does not by itself solve cycles or guarantee thread safety.

Q2. Why might @Transactional appear to do nothing?

Model checkpoint: draw the call path. The target may not be a managed bean, the call may bypass the proxy through self-invocation, the method may not be interceptable under the configured proxy strategy, the wrong transaction manager may be selected, or the observed effect may reflect rollback rules rather than missing interception.

Q3. Is a Spring singleton thread-safe?

Model checkpoint: scope determines identity and lifetime, not thread safety. A singleton is shared by request threads; immutable/stateless design or explicit synchronization is required.

Q4. What does Boot auto-configuration guarantee?

Model checkpoint: it contributes conditional configuration based on documented conditions and normally backs away from specific user configuration. Application code should depend on public configuration contracts, not internal auto-configuration methods.

SDE-2 follow-ups

- 1 A batch job needs one transaction per item and currently calls an annotated method on `this`. Refactor it and explain pool sizing.
- 2 A service must write a row and publish Kafka. Draw failure windows and design an outbox relay.
- 3 A request-scoped identity is needed in an asynchronous task. Explain why capturing a scoped proxy is risky and propose explicit context propagation.
- 4 A library auto-configuration unexpectedly overrides a test bean. Describe the evidence you would collect before changing precedence.

TRY IT | 8. Exercises

- 1 Draw the bean graph for a checkout service with two gateways, a repository, a clock, and a fraud policy. Mark qualifiers and ownership.
- 2 Refactor a constructor cycle between `OrderService` and `InvoiceService` by extracting the invariant-owning use case.
- 3 Write a test matrix for a transaction that updates inventory and inserts an outbox record. Include a constraint violation and simulated relay failure.
- 4 Explain whether a prototype bean injected into a singleton produces a new instance on every method call. Propose two correct alternatives.
- 5 Design an experiment that proves whether retry advice wraps transaction advice or the reverse without depending on log order alone.

RECAP | 9. Summary checklist

- [] Required collaborators are constructor parameters.
- [] Scope matches ownership; singleton state is immutable or deliberately synchronized.
- [] Startup and shutdown resource ownership are paired.
- [] Auto-configuration is diagnosed through conditions and public configuration contracts.
- [] Every annotation-based cross-cutting behavior is explained through the proxy call path.
- [] Transactions wrap business invariants, not arbitrary repository methods.
- [] Remote effects are excluded from local atomicity and made retry-safe.
- [] Rollback, propagation, isolation, and pool behavior have integration tests.

10. Decision labs and model walkthroughs

Lab A: configuration override without ambiguity

A service needs a production `FraudClient`, a local-development stub, and a deterministic test fake. A fragile design scans all three and hopes profile, primary, and bean-name precedence select the intended one. A stronger design defines one application port and one explicit bean per environment boundary:

JAVA

```
// Dependency-requiring Spring configuration; classes are package-private.
@Configuration(proxyBeanMethods = false)
class FraudConfiguration {
    @Bean
    @ConditionalOnMissingBean(FraudClient.class)
    FraudClient httpFraudClient(FraudProperties properties,
                               HttpClient client) {
        return new HttpFraudClient(properties, client);
    }
}

@TestConfiguration(proxyBeanMethods = false)
class FraudTestConfiguration {
    @Bean
    FraudClient deterministicFraudClient() {
        return command -> FraudDecision.accepted("test-policy");
    }
}
```

The test imports its configuration deliberately. The application asserts exactly one `FraudClient`. An auto-configuration condition can provide a default, but application code must not depend on the condition's internal class structure. Test the following matrix:

Context	Expected bean	Evidence
production dependencies and properties	HTTP adapter	narrow context starts; properties validated
user supplies custom adapter	custom adapter	default condition backs away
test imports fake	deterministic fake	no network client constructed
required property absent	startup fails	actionable binding/validation message
two explicit user adapters	startup fails unless qualified	ambiguity is visible, not silently ordered

This is the difference between convenient defaults and implicit behavior. Auto-configuration should reduce wiring, not hide which capability the application receives.

Lab B: transaction boundary under proxy and pool pressure

Suppose `CheckoutService.checkout()` is transactional and invokes a payment provider while the transaction is open. Load testing shows:

TEXT	
request rate:	300/s
payment p95:	800 ms
DB statement time:	25 ms
pool maximum:	60
pool acquisition p95:	2.4 s

The remote wait keeps about $300 * 0.8 = 240$ concurrent operations wanting connections before counting other queries. A 60-connection pool cannot hide that mismatch. Enlarging it to 240 may overload the database and lengthen locks.

Refactor to a durable workflow:

- 1 short transaction creates `PAYMENT_PENDING` order, operation ID, and outbox command;
- 2 worker calls provider outside the database transaction under a deadline;
- 3 second short transaction accepts provider result only from the expected order version/state and appends the next event;
- 4 timeout stores `PAYMENT_UNKNOWN` and schedules reconciliation by operation ID;
- 5 duplicate provider results are idempotent;
- 6 cancellation races through optimistic state transitions.

The price is eventual completion and a richer state machine. The benefit is honest atomicity, bounded pool occupancy, crash recovery, and explicit unknown outcomes.

Lab C: proxy advice matrix

For each call, predict interception before running a test:

Call path	Transaction advice expected?	Reason
controller holds proxied service and calls advised public method	yes under normal proxy configuration	call enters proxy
target method calls another advised method on <code>this</code>	commonly no	direct target call bypasses proxy
object constructed with <code>new</code> outside container	no	no container proxy reference
scheduled adapter calls injected proxied use case	yes if scheduling target and service wiring are managed	call crosses bean boundary
new thread invokes captured target rather than proxy	no/unsafe assumption	reference and transaction context differ
method returns a future and work continues later	only invocation boundary is certain	later work has its own thread/context semantics

Then write one integration test that records transaction-active state or committed rows at each boundary. Avoid making tests depend on proxy class names; test the application-visible effect.

Lab D: rollback and exception translation

Consider a unique order constraint. The persistence adapter should recognize the exact database/driver constraint signal and translate it to `DuplicateExternalOrder`; unexpected connection and syntax failures remain infrastructure errors. The use case maps a duplicate combined with the same idempotency fingerprint to replay, but a different fingerprint to conflict.

Do not translate every `DataIntegrityViolationException` to `409`; it can represent a programming error, null/check/foreign-key violation, or another constraint. Do not use exception message substrings as the only stable classifier. Integration tests assert:

- transaction leaves no partial rows;
- exact known constraint maps to stable domain outcome;
- unknown constraint is not mislabeled as client error;
- rollback occurs even if exception crosses a proxy boundary;
- replay authorization is re-evaluated.

Decision-lab checkpoint

An SDE-2 answer should make the managed reference, advice path, transaction resource, pool slot, and durable state visible. If the explanation relies on "Spring handles it," draw the call and commit sequence until each failure has an owner.

Primary references

- Spring Framework Reference, "The IoC Container":
<https://docs.spring.io/spring-framework/reference/core/beans.html>
- Spring Framework Reference, "Aspect Oriented Programming with Spring":
<https://docs.spring.io/spring-framework/reference/core/aop.html>
- Spring Framework Reference, "Transaction Management":
<https://docs.spring.io/spring-framework/reference/data-access/transaction.html>
- Spring Boot Reference, "Auto-configuration":
<https://docs.spring.io/spring-boot/reference/using/auto-configuration.html>
- Spring Boot Reference, "Externalized Configuration":
<https://docs.spring.io/spring-boot/reference/features/external-config.html>

Version boundary: examples use Java 21 syntax and Jakarta-era Spring APIs. Exact proxy constraints, callback ordering, condition names, defaults, and supported baselines can change between Spring Framework and Spring Boot release lines. Use the documentation selected by the application's dependency BOM. Later-JDK features are not assumed.

SDE-2 CORE CHAPTER

Chapter 2 - Spring MVC and Durable REST Contracts

Learning objectives

After this chapter, you should be able to:

- trace a Servlet-stack Spring MVC request from filters through `DispatcherServlet`, handler mapping, argument resolution, validation, invocation, and message conversion;
- keep transport DTOs, domain models, and persistence entities separate for explicit compatibility and trust boundaries;
- choose HTTP methods and status codes from method semantics rather than habit;
- produce stable RFC 9457 Problem Details responses without leaking internals;
- design bounded offset or keyset pagination with a deterministic order;
- evolve APIs using additive compatibility, explicit version policy, conditional requests, and idempotency keys; and
- explain where authentication, authorization, CSRF, CORS, validation, and output encoding belong.

1. The MVC request lifecycle

Intuition and formal pipeline

Spring MVC on the Servlet stack is a pipeline of distinct responsibilities:

TEXT

```
socket/container
-> servlet filters (security, correlation, encoding)
-> DispatcherServlet
-> HandlerMapping selects handler
-> HandlerAdapter resolves arguments
-> data binding + Bean Validation
-> controller invokes application use case
-> return-value handling
-> HttpResponseMessage serializes body
-> exception resolvers map failures
-> servlet response
```

Interceptors and asynchronous dispatch add branches, but the separation remains useful. A filter surrounds Servlet dispatch and can apply before Spring selects a controller. A controller advice participates in MVC exception and binding handling. A message converter translates bytes and Java values. A service implements a use case. Treating all of them as interchangeable produces security gaps and error inconsistency.

A request should be modeled as an untrusted byte sequence that becomes increasingly trusted only through parsing, syntactic validation, authorization, and domain validation. Successful JSON deserialization proves shape, not permission or business validity.

Recognition and decision rules

Requirement	Correct layer to consider first	Why
authenticate bearer token	security filter chain/resource-server support	applies before protected handler execution
reject malformed JSON	message conversion / exception mapping	no valid DTO exists yet
require nonblank field	DTO validation	transport-level syntactic contract
enforce customer owns account	application/domain authorization	requires business identity and current state
normalize error body	controller advice / error-response policy	one external contract across controllers
measure use-case latency	service boundary observation	controller time can mix parsing and response I/O
attach request correlation	filter plus explicit propagation	must cover all routes and downstream work

Failure walkthrough: "validated" but unauthorized

An endpoint receives { "accountId": "A-7", "amount": 1000 }. Bean Validation proves the string is present and the amount positive. It does not prove the authenticated principal owns A-7. If the repository query uses only the request's account ID, an attacker can substitute another valid identifier. The application must derive or check authorization using the authenticated identity and use a query/transition whose predicate includes the tenant or owner boundary.

2. DTOs are boundary contracts

Separate transport, domain, and entity models

A transport record is optimized for a wire contract. A domain object protects business invariants. A persistence entity participates in an ORM lifecycle. One class rarely satisfies all three without coupling accidental fields and annotations.

JAVA

```
// Dependency-requiring annotations: Jakarta Validation and Spring MVC.
public record CreateOrderRequest(
    @NotBlank String customerId,
    @NotEmpty List<@Valid LineRequest> lines,
    @Size(max = 200) String clientNote) {
}

public record LineRequest(
    @NotBlank String sku,
    @Positive int quantity) {
}

public record OrderResponse(
    String id,
    String status,
    long version,
    Instant createdAt,
    List<LineResponse> lines) {
}
```

Do not bind arbitrary request fields directly onto a managed entity. Over-posting can modify fields the endpoint never intended to expose, and ORM relationships can cause unexpected reads or writes. Map explicitly. Define whether unknown JSON properties are rejected or ignored and test that policy; the serializer's default is not a substitute for an API decision.

Validation has at least four levels:

- 1 **parse/shape:** valid JSON and expected types;
- 2 **syntactic:** length, requiredness, ranges, basic formats;
- 3 **semantic:** SKU exists, transition is allowed, totals match rules;
- 4 **concurrent:** database constraint or atomic predicate still holds at commit.

Cross-field constraints such as `start < end` belong in a type-level validator or command constructor. Database uniqueness must be enforced by a database constraint even if an early validation query improves the error message.

3. HTTP method and status semantics

Method decision table

HTTP semantics are an interoperability contract shared by clients, proxies, caches, and observability systems.

Method	Intended semantics	Typical use	Common trap
GET	safe retrieval; repeated requests should not request a state change	read resource or collection	triggering commands, incrementing durable state as the purpose
HEAD	same selected representation metadata as GET without response content	probes and cache checks	implementing different authorization than GET
POST	resource-specific processing; not inherently idempotent	create subordinate resource or command	assuming retry is safe without an idempotency protocol
PUT	replace/create state at a known target; idempotent intent	client-known resource URI, full replacement	silently treating omitted fields as "unchanged"
PATCH	apply partial modification under the selected patch media type	deliberate partial update	using an undocumented ad-hoc object and losing null/absent distinction
DELETE	remove association/resource; idempotent intent	deletion/tombstone transition	promising physical erasure or immediate downstream disappearance

"Idempotent" means multiple identical requests have the same intended effect as one; responses can differ because state, timestamps, or audit records change. "Safe" means the client did not request a state change, though servers may log or meter.

Status selection

Use the most specific status whose semantics match the outcome:

- **200 OK**: successful request with a representation;
- **201 Created**: a new resource was created; include a **Location** when a resource URI is available;
- **202 Accepted**: processing was accepted but not completed; expose job/status semantics;
- **204 No Content**: success with no response content;
- **304 Not Modified**: conditional retrieval result, not a generic success body;
- **400 Bad Request**: malformed syntax or request contract that cannot be processed as sent;
- **401 Unauthorized**: authentication is missing/invalid; the name is historical;
- **403 Forbidden**: identity is understood but policy denies access;
- **404 Not Found**: no exposed resource; can also avoid disclosing resource existence under a documented policy;
- **409 Conflict**: request conflicts with current resource state, such as an invalid transition or uniqueness collision;
- **412 Precondition Failed**: **If-Match** or another precondition failed;
- **415 Unsupported Media Type**: request representation is unsupported;
- **422 Unprocessable Content**: syntax is understood but instructions fail semantic validation, when this is the API's documented policy;
- **429 Too Many Requests**: rate policy rejected the request, often with retry guidance;
- **500/503**: unexpected server failure versus temporary inability; do not promise retry unless the operation is safe to retry.

Do not return **200** with `{success:false}` for every failure. It defeats generic clients, metrics, caches, and alerting. Do not expose stack traces or SQL messages in any error body.

4. Problem Details as an error protocol

RFC 9457 defines a media type and fields for machine-readable problem details. A useful response might be:

JSON

```
{
  "type": "https://api.example.com/problems/order-version-conflict",
  "title": "Order version conflict",
  "status": 409,
  "detail": "The order changed after the supplied version.",
  "instance": "/orders/0-91",
  "code": "ORDER_VERSION_CONFLICT",
  "traceId": "6f6d...",
  "currentVersion": 8
}
```

The stable machine contract is the problem type URI and documented extension fields. `title` is short and stable. `detail` can help a human but should not be parsed. `traceId` correlates server evidence without exposing internals. Validation errors need a bounded structured collection such as `{path, code, message}`; never echo secrets or whole rejected payloads.

Dependency-requiring Spring sketch:

JAVA

```
@RestControllerAdvice
final class ApiExceptionHandler {
    @ExceptionHandler({VersionConflict.class})
    ResponseEntity<ProblemDetail> versionConflict(
        VersionConflict failure, HttpServletRequest request) {
        ProblemDetail problem = ProblemDetail.forStatus(HttpStatus.CONFLICT);
        problem.setType(URI.create(
            "https://api.example.com/problems/order-version-conflict"));
        problem.setTitle("Order version conflict");
        problem.setDetail("The order changed after the supplied version.");
        problem.setInstance(URI.create(request.getRequestURI()));
        problem.setProperty("code", "ORDER_VERSION_CONFLICT");
        problem.setProperty("currentVersion", failure.currentVersion());
        return ResponseEntity.status(problem.getStatus()).body(problem);
    }
}
```

Exception mapping must preserve operational classification. A client error should not generate a false server-error alert. An unexpected exception should be logged once at the responsible boundary with correlation and then mapped to a generic problem. Avoid catching `Exception` in every controller and duplicating policy.

Failure walkthrough: leaking exception text

If a unique constraint throws a database exception containing an index name and row values, returning `exception.getMessage()` leaks storage details and possibly user data. Mapping every database exception to `409` is also wrong: connection loss or syntax errors are server failures. Translate only recognized constraint outcomes through a narrow persistence adapter; let unexpected failures remain unexpected.

5. Pagination: bounded work and stable order

Pagination contract

Every collection endpoint needs:

- a maximum page size enforced server-side;
- a total deterministic ordering, including a unique tie-breaker;
- filter and sort allow-lists;
- cursor semantics tied to the query and authorization scope;
- a policy for concurrent inserts/deletes;
- opaque cursor validation and expiry/version behavior;
- no promise of an exact total unless the system can afford and define it.

Offset pagination asks the database to skip **offset** rows. It is simple and supports page numbers, but deep offsets can do increasing work and concurrent changes can shift results. Keyset pagination asks for rows after the last ordering key:

SQL

```
select id, created_at, status
from orders
where customer_id = :customer
  and (created_at, id) < (:last_created_at, :last_id)
order by created_at desc, id desc
limit :limit_plus_one;
```

The **(created_at, id)** tuple establishes a total order. Fetching one extra row tells the API whether a next cursor exists. The exact SQL tuple support and plan are database-specific; an equivalent lexicographic predicate may be required.

Cursor design

A cursor can encode a version, last sort keys, normalized filter hash, and perhaps expiry, then be authenticated or otherwise tamper-resistant:

TEXT

```
v1 | createdAt=... | id=... | filterHash=... | signature=...
```

Opaque does not mean secure; base64 is merely encoding. Reject a cursor created for a different tenant or filter. Do not place secrets in it. If records mutate their ordering field, even keyset pagination can skip or duplicate items; document whether the listing is a live traversal or backed by a snapshot.

Decision rule

- Choose offset for small administrative lists where page numbers and exact jumping matter.
- Choose keyset for large feeds or APIs where forward/backward traversal and predictable query work matter.
- Choose a snapshot/export job when the caller needs a stable exhaustive dataset.

6. Conditional requests and optimistic HTTP concurrency

An entity tag identifies a selected representation. A client can send **If-None-Match** on GET to avoid transferring an unchanged representation. For updates, **If-Match** can make the request conditional on the observed version.

TEXT

```
GET /orders/0-91
-> 200, ETag: "order-91-v8"

PUT /orders/0-91
If-Match: "order-91-v8"
-> 204 and new version, or 412 if the precondition fails
```

Connect the precondition to an atomic persistence predicate; checking in Java and updating later creates a race. Distinguish weak and strong validators according to HTTP semantics. Do not invent an ETag from a Java object's `hashCode()` -it is not a durable representation validator.

7. API evolution and versioning

Compatibility before version numbers

Prefer additive evolution:

- add optional response fields that tolerant clients can ignore;
- add request fields with safe defaults only when absence is unambiguous;
- preserve enum handling policy or provide an unknown-value strategy;
- never silently change units, timezone meaning, nullability, precision, or ordering;
- announce and observe deprecation before removal;
- test representative old clients/contracts.

Use a new version when semantics cannot remain compatible. URI, media-type, header, or query versioning each has operational tradeoffs; consistency matters more than fashion. The version selector must participate in routing, documentation, metrics, caching (**Vary** where appropriate), and deprecation policy.

Schema and behavior versions are not always identical. A field can retain its JSON shape while its meaning changes incompatibly. Write semantic examples and consumer contract tests.

8. The security boundary

Security is not one annotation. The request crosses several controls:

- 1 TLS protects transport in deployment;
- 2 the security chain authenticates credentials and establishes a principal;
- 3 route and method policies authorize broad capabilities;
- 4 the use case enforces resource/tenant ownership and state-dependent policy;
- 5 input is constrained for resource use and injection resistance;
- 6 output is encoded for its destination and secrets are redacted;
- 7 logs and metrics avoid sensitive values;
- 8 rate and concurrency limits protect capacity.

CORS tells a browser whether cross-origin script access is permitted; it is not authentication. CSRF defenses matter when a browser automatically attaches credentials such as cookies. A stateless bearer-token design changes the threat model but does not eliminate authorization, token theft, replay, or origin concerns. Use the supported Spring Security model and test deny-by-default behavior.

Mass-assignment, insecure direct object reference, unrestricted page sizes, regex denial of service, decompression bombs, and oversized payloads are API-boundary failures even when deserialization succeeds.

TRY IT | 9. Interview questions and model checkpoints

Q1. Trace a request to a controller.

Model checkpoint: begin with container/filter chain, then `DispatcherServlet`, mapping/adapter, argument resolution and conversion, validation, handler invocation, return-value/message conversion, and exception resolution. Identify where security applies and avoid asserting undocumented internal order.

Q2. When do you choose 409 versus 412?

Model checkpoint: 412 is specifically a failed request precondition such as `If-Match`; 409 represents a conflict with current resource state when that precondition protocol is not the cause.

Q3. Why can keyset pagination be more stable than offset?

Model checkpoint: the next query continues from explicit ordered keys instead of skipping a moving row count. It needs a total order and cannot offer arbitrary page jumps cheaply. Mutable ordering keys and live writes still require documented semantics.

Q4. Is validation authorization?

Model checkpoint: no. Validation checks input or domain rules; authorization checks whether this principal may act on this resource under current policy. Both are necessary.

SDE-2 follow-ups

- 1 Design error types that remain compatible across ten services while allowing service-specific extensions.
- 2 An API times out after creating an order. Define client retry behavior and the server protocol that prevents duplicates.
- 3 Add backward pagination to a descending (`createdAt`, `id`) feed. Explain predicates and response ordering.
- 4 Explain how API version selection affects CDN/proxy caches and observability dimensions.

TRY IT | 10. Exercises

- 1 Specify request, success, validation, conflict, and unexpected-error contracts for `POST /orders`.
- 2 Convert an unbounded `GET /events` into keyset pagination. Define the cursor payload, maximum limit, tie-breaker, and behavior under deletion.
- 3 Design a conditional update using an ETag and an atomic SQL version predicate.
- 4 Threat-model an endpoint that exports a CSV using a user-provided sort column and filename.
- 5 Write five contract tests that distinguish malformed JSON, validation failure, forbidden access, missing resource, and version conflict.

RECAP | 11. Summary checklist

- ☐ The request lifecycle is understood as stages with distinct ownership.
- ☐ DTOs are separate from domain and persistence entities.
- ☐ Validation includes syntactic, semantic, authorization, and concurrent guards.
- ☐ Methods and statuses preserve HTTP semantics.
- ☐ Problems expose stable machine codes without internal details.
- ☐ Collection work is bounded and ordered deterministically.
- ☐ Cursor and conditional-request contracts are explicit.
- ☐ API changes follow a compatibility and deprecation policy.
- ☐ Authentication, authorization, CSRF/CORS, and resource controls are not conflated.

12. API contract review lab

Review this tempting endpoint:

JAVA

```
// Dependency-requiring anti-example.  
@PostMapping("/orders/search")  
List<OrderEntity> search(@RequestBody Map<String, Object> body) {  
    return repository.search(body);  
}
```

It exposes persistence entities, has no page bound, accepts untyped filters, hides authorization, and has undocumented serialization/error behavior. It also uses POST for a read without explaining why a body is required or how caches/clients should treat it.

A contract-first revision defines:

TEXT

```
GET /v1/orders?status=PENDING&createdBefore=...&limit=50&after=...  
Authorization derived tenant/customer scope  
Accept: application/json  
  
200 {  
  "items": [...stable OrderSummary fields...],  
  "nextCursor": "opaque-or-null"  
}
```

If filters are too complex for practical query parameters, a POST search resource can be valid, but document safety/idempotency, cacheability, maximum complexity, and whether the request creates an asynchronous export. Method choice follows semantics and interoperability, not a universal ban.

Contract table

Concern	Decision	Test evidence
scope	tenant/customer comes from authenticated principal	cross-tenant ID/filter returns no data
filter	enum and timestamp parsed with allow-list	unknown status and invalid instant produce stable problems
sort	fixed newest-first (<code>createdAt, id</code>)	ties traverse without skip/duplicate
bound	default 25, maximum 100	0, negative, 101, huge value rejected or normalized per contract
cursor	signed/versioned, binds tenant+filter	tamper and cross-filter reuse rejected
projection	stable <code>OrderSummary</code> , not entity	lazy fields/PII never serialize
total	omitted	client does not infer final page from page length alone; cursor signals continuation
errors	RFC 9457 types	status, media type, code, no internal message
cache	private/no-store or tenant-safe explicit policy	Vary and authorization cache behavior reviewed

DTO mapping walkthrough

The controller resolves `PrincipalContext`, parses a bounded filter, and invokes `ListOrdersQuery`. The query constructs a database predicate that includes tenant scope and cursor tuple, selects only summary columns, requests `limit + 1`, and returns a value object. The adapter maps the first `limit` rows and generates a cursor from the last returned row only if an extra row exists.

If the cursor contains (`createdAt, id`) but not the status filter, a client can change `status` mid-traversal and receive surprising data. Bind normalized query shape in the cursor or explicitly reject mismatch. If tenant ID is merely encoded and not authenticated, a caller can rewrite it. Use an authenticated cursor or ignore embedded authorization identity and always derive it server-side.

Error taxonomy exercise

Create a stable mapping:

Failure	HTTP outcome	Retry guidance
malformed JSON/query encoding	400 problem	fix request
field validation	400 or documented 422	fix fields
unauthenticated credential	401	authenticate/refresh under security protocol
forbidden capability	403	do not blind retry
resource hidden/missing	404	re-check identity; no existence disclosure
stale If-Match	412	fetch/reconcile current representation
invalid domain transition	409	change requested action/state
rate policy	429	wait only if safe; honor bounded guidance
dependency unavailable	503 generic problem	retry only idempotent operation with budget
unexpected defect	500 generic problem	server investigation; no internal detail

Whether semantic validation is 400 or 422 is less important than a consistent documented policy. A client must never parse localized prose to decide.

Conditional-update race walkthrough

Two clients read version 8. Client A sends **If-Match: "v8"** and atomically updates to version 9. Client B also sends **v8**; its SQL predicate matches zero and the API returns 412. If the service instead checks version 8 with a SELECT, calls a remote service, then updates without version in the predicate, B can overwrite A. The HTTP precondition has value only when connected to the authoritative atomic transition.

Versioning checkpoint

Before creating **/v2**, classify the proposed change:

- adding optional response metadata: usually additive;
- changing cents to decimal currency units: semantic breaking change;
- removing an enum value clients may send: breaking request change;
- adding a required request field without safe default: breaking;
- changing default sort: behavior breaking even if JSON schema unchanged;
- tightening an unsafe maximum page size: potentially necessary security change; communicate and measure;
- replacing offset with cursor while keeping old contract during migration: parallel capability/deprecation.

Versioning does not eliminate compatibility work. Old versions still need security fixes, telemetry, data mapping, and decommission evidence.

Primary references

- Spring Framework Reference, "Spring Web MVC": <https://docs.spring.io/spring-framework/reference/web/webmvc.html>
- Spring Framework Reference, "Error Responses": <https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-ann-rest-exceptions.html>
- Spring Security Reference: <https://docs.spring.io/spring-security/reference/>
- RFC 9110, "HTTP Semantics": <https://www.rfc-editor.org/rfc/rfc9110>
- RFC 9111, "HTTP Caching": <https://www.rfc-editor.org/rfc/rfc9111>
- RFC 9457, "Problem Details for HTTP APIs": <https://www.rfc-editor.org/rfc/rfc9457>

Version boundary: the architectural pipeline and HTTP RFC semantics are the baseline. Concrete Spring MVC argument resolvers, validation integration, `ProblemDetail` conveniences, and security DSLs vary across supported framework lines. Examples are Jakarta-era and use Java 21; consult the documentation matched to the deployed BOM.

SDE-2 CORE CHAPTER

Chapter 3 - Idempotent APIs, Operational Boundaries, and Spring Testing

Learning objectives

After this chapter, you should be able to:

- distinguish naturally idempotent state replacement from an idempotency-key protocol for non-idempotent commands;
- design an atomic request-deduplication record with payload binding and replay semantics;
- reason about deadlines, cancellation, retries, duplicate execution, and overload as one reliability problem;
- expose Spring Boot Actuator information without turning management endpoints into a data leak or control-plane vulnerability;
- choose unit, MVC slice, persistence slice, container, and end-to-end tests from the claim under test; and
- build a production-readiness checklist that includes shutdown, probes, configuration, metrics, and security.

1. Idempotency is a server-side state machine

Intuition and formal contract

Networks can lose a response after the server commits. A client sees a timeout but cannot infer whether execution occurred. Retrying a non-idempotent **POST** can therefore duplicate the effect.

An idempotency-key protocol binds a client-generated operation identity to one logical request. For principal/tenant **T**, endpoint operation **O**, key **K**, and canonical payload fingerprint **H**, the server maintains a record:

TEXT

```
(T, O, K) -> {fingerprint H, state, outcome reference, expiry}  
state in {IN_PROGRESS, SUCCEEDED, FAILED_RETRYABLE?, FAILED_FINAL}
```

A useful contract is:

- 1 the first accepted **(T,O,K,H)** reserves the key atomically;
- 2 the business result and deduplication outcome become durable in one appropriate transaction, or a recoverable state machine connects them;
- 3 the same key with a different fingerprint is rejected;

- 4 the same key and fingerprint replays the documented prior outcome or directs the client to its status;
- 5 concurrent duplicates do not both execute the effect;
- 6 retention is longer than the documented client retry window;
- 7 authorization is evaluated for every request, including replay.

The key is scoped. A global key without tenant and operation context can collide or leak another caller's result. A payload fingerprint must use a documented canonicalization; hashing raw JSON makes whitespace or property order significant, while careless normalization can merge meaningfully different requests.

Recognition and decision rule

Use an idempotency key when clients may retry a command that creates a new server-chosen identity or produces a costly/non-repeatable side effect. Prefer a naturally idempotent resource design when possible: `PUT /imports/{clientGeneratedId}` can make the target identity explicit. Do not add idempotency storage to a pure `GET` merely because every API should "have idempotency."

Concrete Spring/JDBC sketch - dependency-requiring

JAVA

```
@Transactional
public CreateOrderResult create(
    TenantId tenant,
    String key,
    CreateOrderCommand command) {
    String fingerprint = canonicalFingerprint(command);
    IdempotencyRecord record = idempotency.reserveOrRead(
        tenant, "create-order", key, fingerprint);

    if (record.completed()) {
        return orders.resultFor(record.resourceId());
    }
    if (!record.ownedByThisAttempt()) {
        throw new OperationInProgress(record.retryAfter());
    }

    Order order = Order.create(command);
    orders.insert(order);
    outbox.append(OrderCreated.from(order));
    idempotency.complete(record.id(), order.id());
    return CreateOrderResult.created(order);
}
```

The repository needs a unique constraint on the scoped key. `reserveOrRead` must handle the insert race. The database transaction connects the order, outbox event, and completed key. If external authorization must happen first, do it without holding locks. If payment must occur remotely, the workflow needs a recoverable saga/state machine; a local transaction cannot include the provider.

Failure walkthroughs

Response lost after commit: the retry finds **SUCCEEDED** and returns the same logical result. This is the central success case.

Two requests arrive concurrently: a unique constraint/atomic insert gives one ownership; the other reads **IN_PROGRESS** and waits, polls, or receives a documented conflict/retry response. A "check then insert" without a unique constraint races.

Same key, changed body: return a client error. Reusing the previous result would associate the wrong intent with the key.

Process dies after reservation but before business commit: if reservation and work share one transaction, both roll back. If they cannot, store a lease/attempt token and define recovery. Never leave permanent **IN_PROGRESS** records with no reclamation policy.

Retention expires too soon: a late retry becomes a new operation. The API must document its deduplication window; longer retention has storage and privacy costs.

2. Deadlines, retries, and overload

One end-to-end budget

A timeout at each hop is not a coherent deadline. If an inbound request has budget **D**, downstream work must receive a smaller remaining budget after local queueing and processing:

TEXT

```
remaining = requestDeadline - now
downstreamTimeout <= remaining - responseAndSafetyMargin
```

Retries consume time and amplify load. For attempt cost **t**, backoffs **b_i**, and **n** attempts, worst-case elapsed time approximates $\sum(t_i) + \sum(b_i)$. Retry only transient failures, only when the operation is idempotent or protected by a deduplication contract, and only within a bounded attempt and elapsed-time budget. Add jitter to reduce synchronized retries.

Cancellation is cooperative. A client disconnect or timeout does not prove database, broker, or remote work stopped. Propagate interruption/deadline where supported, but design duplicate safety because cancellation races with completion.

Capacity controls

- **rate limit:** bounds admitted work over time for a caller or class;
- **concurrency limit:** bounds simultaneous in-flight work;
- **queue bound:** limits waiting work and memory;
- **bulkhead:** partitions capacity so one dependency/tenant cannot consume all of it;
- **circuit breaker:** temporarily rejects calls when evidence suggests the dependency cannot succeed;
- **backpressure:** communicates or enforces that producers must slow down.

Reject early when the system cannot meet the deadline. An unbounded servlet executor queue converts overload into high latency and heap pressure. Virtual threads can reduce thread-per-request cost for blocking code but do not expand database connections, remote capacity, memory, or CPU.

3. Actuator and production-safe observability

Management endpoints are an API

Spring Boot Actuator can expose health, metrics, loggers, environment, mappings, thread information, and other management capabilities depending on dependencies and configuration. Treat every exposed endpoint as a privileged operational API. Use an explicit exposure allow-list, separate network/security boundary when appropriate, authentication/authorization, TLS, and data redaction. Never assume an endpoint is safe because it is "internal."

Useful distinctions:

- **liveness:** should the orchestrator restart this process? It should not fail merely because an optional dependency is down, or every instance may restart together.
- **readiness:** should this instance receive new traffic? It can reflect inability to serve required work.
- **startup:** has initialization completed within its allowed window?
- **deep diagnostic health:** detailed dependency evidence for operators; not necessarily a public traffic probe.

Probe behavior must match deployment. A readiness check that runs a costly query every second can become load. A cached check can hide fast failure. A database-down response may correctly make an instance unready, but if every instance does that while work could degrade, the policy creates total outage.

Metrics and cardinality

Prefer bounded labels such as route templates, method, outcome, and dependency name. Never use raw URL, user ID, order ID, exception message, or idempotency key as a metric tag. High cardinality consumes memory and makes backends expensive. Put per-request identity in structured logs or traces with controlled sampling and retention.

Instrument the use-case and dependency boundaries:

TEXT

```
request rate, error rate, duration distribution
in-flight requests and admission rejections
connection-pool active/idle/wait/timeout
outbox backlog age and count
idempotency conflicts and replay count
downstream latency/errors/timeouts
JVM allocation, GC, CPU, and thread/virtual-thread evidence
```

Histograms support percentiles and SLO analysis; averages hide tail latency. Define which clock and units are used. A trace is not a replacement for metrics: tracing explains selected executions, while metrics reveal aggregate change.

4. Graceful startup and shutdown

Readiness should become false before shutdown stops accepting work. Allow in-flight requests a bounded drain period. Stop message consumption, finish or abandon work under the consumer protocol, flush recoverable state, and close resources. Shutdown hooks have a deadline imposed by the platform; a design that requires infinite cleanup is not graceful.

Background executors must have explicit ownership. Avoid fire-and-forget `CompletableFuture` work on an unspecified common pool. Name the executor, bound or control admission, propagate only required context, observe failures, and stop it. Request-scoped state stored in thread locals does not automatically follow asynchronous work and can leak if not cleared.

5. Testing by claim

Test pyramid as an evidence matrix

Claim	Smallest credible test	What it cannot prove
domain transition rejects invalid status	plain JUnit test, real domain object	MVC mapping, transaction, database behavior
controller maps DTO and status correctly	MVC slice with security behavior explicit	production database or full auto-configuration
JPA mapping/query works	persistence slice or focused integration test against target engine	HTTP contract and full transaction orchestration
custom configuration conditions select a bean	application-context runner/narrow context	complete production startup
rollback and unique constraint protect duplicates	integration test with real transaction manager and target DB	deployment routing and proxy behavior outside test config
authorization denies cross-tenant access	security-enabled MVC/integration test	network gateway policy
entire critical path works	end-to-end application test	exhaustive business combinations

"Slice" tests intentionally include only part of the application. A missing bean may be correct for the slice rather than an application bug. State the slice's boundary. Avoid `@MockBean`-style replacement for every collaborator until the test merely verifies a mock script.

MVC contract test - dependency-requiring

JAVA

```
@WebMvcTest(OrderController.class)
class OrderControllerTest {
    @Autowired MockMvc mvc;
    @MockBean CreateOrderUseCase useCase;

    @Test
    void rejectsMissingIdempotencyKey() throws Exception {
        mvc.perform(post("/orders")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                {"customerId":"C-1","lines":[{"sku":"S-1","quantity":1}]}
                """))
            .andExpect(status().isBadRequest())
            .andExpect(content().contentTypeCompatibleWith(
                MediaType.APPLICATION_PROBLEM_JSON));
    }
}
```

This test proves mapping under the selected slice configuration. It does not prove the unique key or transaction behavior. For that, send two concurrent requests or call the real service against the real database engine, then assert exactly one order and one outbox record.

Integration-test discipline

- isolate state with transaction rollback only when rollback matches production behavior under test;
- do not wrap a test in a transaction when you need to observe after-commit behavior;
- control time through an injected `Clock`;
- use deterministic IDs when identity itself is not under test;
- test the target database for locking, isolation, SQL, and query plans;
- assert persisted outcomes, not only returned values;
- run concurrent tests repeatedly with barriers to create the race window;
- keep external service stubs protocol-faithful, including timeouts and duplicate delivery;
- record why each full-context test needs the full context.

TRY IT | 6. Interview questions and model checkpoints

Q1. How do idempotency keys differ from optimistic locking?

Model checkpoint: an idempotency key deduplicates repeated command intent; optimistic locking detects that a resource changed since a version was observed. A create command may need the former, while an update may use `If-Match` plus a version predicate. They solve different races and can coexist.

Q2. What should a readiness check verify?

Model checkpoint: only conditions required to admit new traffic under the deployment's service policy, cheaply and without causing failure amplification. Liveness asks whether restart can help; detailed diagnostics belong behind an operator boundary.

Q3. When is a full Spring context test justified?

Model checkpoint: when the claim depends on real auto-configuration, proxy advice, security chain, serialization configuration, or cross-layer wiring. Pure decisions should remain plain unit tests.

Q4. Why can retries worsen an outage?

Model checkpoint: each failed request creates more attempts, increasing queueing and dependency load. Retry budgets, backoff with jitter, admission controls, and circuit breaking limit amplification; idempotency protects correctness but not capacity.

SDE-2 follow-ups

- 1 An idempotency request is stuck **IN_PROGRESS** after a host crash. Design lease ownership and recovery without executing twice.
- 2 An Actuator metrics endpoint causes memory growth. Identify label cardinality and observation leaks.
- 3 A controller test passes while production rejects every request with **403**. Enumerate security configuration differences the slice may have hidden.
- 4 Design a graceful shutdown protocol for an HTTP service that also consumes messages and owns an outbox relay.

TRY IT | 7. Exercises

- 1 Specify the table schema and state transitions for a 24-hour idempotency window. Include tenant scope, fingerprint mismatch, crash recovery, and data erasure.
- 2 Create a deadline budget for a 750 ms API with two downstream calls and at most one retry. State the rejection policy.
- 3 Classify ten Actuator endpoints into exposed, operator-only, or disabled for an internet-facing service, and justify each.
- 4 Write a test plan that proves two concurrent identical create requests produce one logical order.
- 5 Diagnose why a readiness probe tied to a third-party analytics API caused every instance to leave service.

8. Production-readiness checklist

- ☐ Idempotency keys are scoped, payload-bound, atomic, retained for a documented window, and authorized on replay.
- ☐ Deadlines and retries share one bounded budget.
- ☐ Queue, rate, and concurrency limits protect every scarce dependency.
- ☐ Management endpoints use an explicit exposure and security policy.
- ☐ Health groups reflect orchestration semantics, not a list of every dependency.
- ☐ Metric dimensions are bounded; logs and traces are redacted.
- ☐ Background tasks have named ownership, failure observation, and shutdown.
- ☐ Each test uses the smallest environment that can prove its claim.
- ☐ Database and concurrency guarantees are tested against production-relevant infrastructure.

9. Operational security drill

Assume the public gateway blocks `/actuator/**`, but a misconfigured alternate ingress reaches the application port directly. "The gateway protects it" is not defense in depth. The application exposure allow-list should still omit dangerous endpoints, management access should have its own authenticated network/policy boundary, and deployment tests should scan every reachable service/port.

Test from three perspectives:

- 1 public internet identity cannot reach management routes or infer environment details;
- 2 operator identity reaches only the endpoints required by role;
- 3 orchestrator probes reach a minimal health path without receiving component secrets;
- 4 application logs do not include property values, token headers, heap snippets, or request bodies from failed management authentication;
- 5 emergency changes to log level or shutdown behavior are authenticated, authorized, audited, rate-limited, and disabled when unnecessary.

Health responses should reveal only what the caller needs. A public `503` can say temporarily unavailable; it need not name the database host. An operator endpoint can expose component status under policy, but secrets and personal data remain redacted.

Then simulate management-plane slowness. Scraping a metric endpoint must not exhaust request threads or lock the application. Expensive diagnostics such as heap/thread information are operator actions, not high-frequency probes. Monitor telemetry export/drop behavior so an observability outage does not become an application outage or a false "all healthy" signal.

Checkpoint: list the network path, Spring Security chain, Actuator exposure list, role policy, payload redaction, audit record, and resource limit. Security exists at every reachable path, not only the diagram's preferred gateway.

Primary references

- Spring Boot Reference, "Production-ready Features": <https://docs.spring.io/spring-boot/reference/actuator/>
- Spring Boot Reference, "Endpoints": <https://docs.spring.io/spring-boot/reference/actuator/endpoints.html>
- Spring Framework Reference, "Testing": <https://docs.spring.io/spring-framework/reference/testing.html>
- Spring Boot Reference, "Testing": <https://docs.spring.io/spring-boot/reference/testing/>
- RFC 9110, "HTTP Semantics": <https://www.rfc-editor.org/rfc/rfc9110>
- RFC 9457, "Problem Details for HTTP APIs": <https://www.rfc-editor.org/rfc/rfc9457>

Version boundary: exact Actuator endpoints, exposure defaults, health-group options, test annotations, replacement-test-bean APIs, and graceful-shutdown settings vary by Boot release line. The design rules above do not assume an undocumented default. Check the BOM-matched reference. Java examples target Java 21.

SDE-2 CORE CHAPTER

Chapter 4 - Worked Spring Service: A Retry-Safe Order API

1. Problem statement and requirements

Design a Java 21 Spring Boot service that lets an authenticated customer create and read orders.

Functional requirements:

- `POST /v1/orders` creates one logical order even if a client retries;
- each order contains 1-100 SKU/quantity lines;
- creation reserves inventory in the same relational database;
- downstream fulfillment receives an `OrderCreated` event;
- `GET /v1/orders/{id}` returns only an order visible to the caller;
- `GET /v1/orders` provides bounded newest-first traversal;
- `PUT /v1/orders/{id}/note` updates a note conditionally;
- failures use a documented Problem Details contract.

Nonfunctional requirements:

- 99.9% monthly availability objective for the API's defined good-event criteria;
- a 500 ms server deadline for ordinary create/read requests, excluding asynchronous fulfillment;
- no overselling under concurrent requests;
- horizontally scalable stateless application instances;
- auditable state transitions without secrets or full payloads in logs;
- safe rolling restart and observable outbox lag.

Clarify during an interview: inventory and orders are in one database; payment is outside this version; fulfillment accepts duplicate delivery and supports event-ID deduplication; immediate global read-after-write through arbitrary replicas is not promised.

2. Architecture and ownership

TEXT

```

client
  -> gateway/TLS/rate policy
  -> Spring Security filter chain
  -> OrderController (transport mapping)
  -> CreateOrderUseCase (business + transaction boundary)
  -> OrderRepository -----+
  -> InventoryRepository -+---> relational DB
  -> IdempotencyRepository+
  -> OutboxRepository -----+

outbox relay -> broker -> fulfillment consumer

Actuator/metrics/logs/traces -> restricted operations plane

```

The controller owns HTTP translation. The use case owns the invariant and local transaction. Repositories own persistence translation. The outbox relay owns eventual publication. Security establishes identity at the edge; the use case still scopes every resource access to the tenant/customer.

The bean graph is deliberately acyclic:

TEXT

```

OrderController -> CreateOrderUseCase -> repository ports
                  -> ReadOrderQuery   -> read repository
OutboxRelay      -> outbox repository + event publisher

```

The relay does not call the controller or use case. The use case does not know Spring MVC. This supports plain-Java domain tests and narrow integration tests.

3. API contract

Create

HTTP

```

POST /v1/orders
Authorization: Bearer ...
Idempotency-Key: 3f4d...
Content-Type: application/json

{
  "lines": [
    {"sku": "B00K-21", "quantity": 2}
  ],
  "clientNote": "Leave at reception"
}

```

First successful creation returns **201 Created**, a **Location**, an ETag derived from the order version, and a representation. A replay using the same key and equivalent canonical request returns the same logical resource and a documented replay indicator; whether the status remains **201** or becomes **200** is part of the

API contract. A key reused with a different payload returns a client error problem. A concurrent request may wait briefly or return a documented operation-in-progress problem.

Do not accept `customerId` from the body. Derive it from the authenticated principal. Limit header length and body size before expensive parsing.

Read and list

HTTP

```
GET /v1/orders/0-91
If-None-Match: "order-0-91-v8"
```

Return **304** if the selected representation validator matches. Query by both order ID and customer/tenant boundary so a guessed identifier cannot expose another customer's record.

HTTP

```
GET /v1/orders?limit=50&after=eyJ2IjoxLCJhZnRlciI6...
```

Limit defaults to 25 and is capped at 100. Ordering is `(created_at DESC, id DESC)`. The opaque authenticated cursor binds the last tuple, tenant, filter, and cursor schema version. The response contains items and `nextCursor`; it does not promise an exact total.

Conditional note update

HTTP

```
PUT /v1/orders/0-91/note
If-Match: "order-0-91-v8"
Content-Type: application/json

{"note":"Ring the bell"}
```

This is replacement of the note subresource. An atomic update predicate includes order ID, owner, and expected version. Return **204** with the new ETag, **412** for a failed precondition, **404** under the resource-disclosure policy, and a validation problem for an oversized note.

4. Domain and persistence contracts

Domain transition

JAVA

```
final class Order {
    private final OrderId id;
    private final CustomerId customerId;
    private final List<OrderLine> lines;
    private OrderStatus status;
    private long version;

    public static Order place(
        OrderId id, CustomerId customerId, List<OrderLine> lines) {
        if (lines.isEmpty() || lines.size() > 100) {
            throw new InvalidOrder("line count must be 1..100");
        }
        // Defensively copy and validate each line.
        return new Order(id, customerId, List.copyOf(lines),
            OrderStatus.PENDING, 0);
    }
}
```

This example omits mapping details but keeps invariants independent from MVC and JPA. The persistence adapter can map to rows/entities. Equality of domain identifiers is value-based; a mutable ORM entity needs a separately considered equality strategy.

Database constraints

Illustrative relational constraints:

SQL

```
create table orders (
  id varchar(40) primary key,
  customer_id varchar(40) not null,
  status varchar(24) not null,
  note varchar(200),
  version bigint not null,
  created_at timestamp with time zone not null
);

create index orders_customer_created_id
  on orders (customer_id, created_at desc, id desc);

create table idempotency_request (
  customer_id varchar(40) not null,
  operation varchar(40) not null,
  request_key varchar(128) not null,
  fingerprint varchar(128) not null,
  state varchar(24) not null,
  order_id varchar(40),
  expires_at timestamp with time zone not null,
  primary key (customer_id, operation, request_key)
);

create table outbox_event (
  event_id varchar(40) primary key,
  aggregate_id varchar(40) not null,
  event_type varchar(80) not null,
  payload text not null,
  occurred_at timestamp with time zone not null,
  published_at timestamp with time zone
);
```

The exact types, descending-index semantics, and query plans are database-specific. Migrations-not ORM auto-DDL in production-own schema evolution. Add foreign keys/check constraints appropriate to the chosen model.

Inventory reservation

Avoid "read quantity, subtract in Java, write value." Use an atomic conditional update:

SQL

```
update inventory
set available = available - :quantity,
    version = version + 1
where sku = :sku
and available >= :quantity;
```

Update count zero means missing or insufficient inventory; translate it through a deliberate query or domain policy. For several lines, reserve in a deterministic SKU order to reduce deadlock opportunities, keep the transaction short, and retry only classified transient transaction failures within the deadline.

5. Spring adapter sketch - dependency-requiring

JAVA (continued 1/2)

```
@RestController
@RequestMapping("/v1/orders")
final class OrderController {
    private final CreateOrderUseCase create;
    private final ReadOrderQuery read;

    OrderController(CreateOrderUseCase create, ReadOrderQuery read) {
        this.create = create;
        this.read = read;
    }

    @PostMapping
    ResponseEntity<OrderResponse> create(
        @AuthenticationPrincipal Jwt principal,
        @RequestHeader("Idempotency-Key") String key,
        @Valid @RequestBody CreateOrderRequest request) {
        CustomerId customer = CustomerId.fromSubject(principal.getSubject());
        CreateOrderResult result = create.execute(
            customer, key, OrderMapper.toCommand(request));
        return ResponseEntity.created(result.location())
            .eTag(result.eTag())
            .header("Idempotency-Replayed",
```

JAVA (continued 2/2)

```
        Boolean.toString(result.replayed()))
        .body(OrderMapper.toResponse(result.order()));
    }

    @GetMapping("/{id}")
    ResponseEntity<OrderResponse> get(
        @AuthenticationPrincipal Jwt principal,
        @PathVariable String id,
        @RequestHeader(value = "If-None-Match", required = false)
            String ifNoneMatch) {
        // A helper compares validators according to HTTP rules.
        OrderView order = read.requireVisible(
            CustomerId.fromSubject(principal.getSubject()),
            new OrderId(id));
        if (EntityTags.matches(ifNoneMatch, order.eTag())) {
            return ResponseEntity.status(HttpStatus.NOT_MODIFIED)
                .eTag(order.eTag()).build();
        }
        return ResponseEntity.ok().eTag(order.eTag())
            .body(OrderMapper.toResponse(order));
    }
}
```

The example shows boundaries, not copy-paste production code. Validate key syntax and ID length. Review framework utilities for correct entity-tag parsing. Avoid hand-parsing authentication tokens in a controller. Do not trust a raw subject without issuer/audience/signature validation in the configured resource server.

TRACE IT | 6. End-to-end execution walkthrough

Happy path

- 1 Gateway admits the request under per-customer and global capacity policy.
- 2 Security verifies the credential and produces an authenticated principal.
- 3 MVC converts JSON into a request DTO and validates syntactic constraints.
- 4 Controller derives `CustomerId`, validates the idempotency-key format, and maps to a command.
- 5 Transactional use case computes a canonical fingerprint.
- 6 Idempotency repository reserves the scoped key under a unique constraint.
- 7 Domain validates line rules.
- 8 Inventory rows are conditionally updated in deterministic order.
- 9 Order and lines are inserted.
- 10 Outbox event is inserted.
- 11 Idempotency record is completed with the order ID.
- 12 Local transaction commits.
- 13 Response returns `201`, `Location`, representation, and ETag.
- 14 Independently, relay claims the outbox event, publishes it, and records publication progress.
- 15 Fulfillment deduplicates by event ID and performs its own transition.

Failure matrix

Failure point	Client observation	Durable state	Recovery
malformed JSON	400 problem	none	fix request; new or same key has not been reserved
insufficient inventory	conflict/semantic problem	transaction rolls back	caller changes order; key policy must permit/document final failure
DB deadlock victim before commit	transient failure or bounded internal retry	transaction rolled back	retry only within budget using same key
response lost after commit	timeout	order, outbox, completed key exist	retry same key replays same result
relay crashes before publish	create already successful	unpublished outbox exists	another relay attempt claims it
relay crashes after publish before mark	create successful; duplicate event possible	outbox still appears pending	republish; consumer deduplicates
fulfillment unavailable	API create may remain healthy	backlog grows	alert on age, retry with backoff, apply backlog controls
process receives shutdown	readiness false	committed work preserved	drain bounded requests, stop relay/consumer safely

The relay cannot obtain exactly-once side effects simply by marking a row. The publish/mark boundary can duplicate. At-least-once delivery plus idempotent consumption is the explicit contract.

7. Security and abuse review

Threats and controls

- **object-ID guessing:** scope repository operations by authenticated customer; consistent disclosure policy;
- **token confusion:** validate signature, issuer, audience, time claims, and algorithm through supported security configuration;
- **payload/resource exhaustion:** request-body, line-count, field-length, rate, concurrency, and database-statement limits;
- **idempotency-key abuse:** length/character restrictions, tenant scoping, retention limits, no key values in metrics;
- **SQL injection:** parameterized values and sort/filter allow-lists;
- **mass assignment:** explicit request DTO and mapper;
- **sensitive logs:** log order ID, outcome code, latency, and trace ID-not tokens, address/note, or raw body;
- **management exposure:** separate network/auth policy and minimum endpoint allow-list;
- **dependency compromise:** locked and reviewed dependency graph, signed/verified build inputs as organizational policy permits.

Authorization belongs in more than the route declaration. A method-security check that loads an order and a later repository update using only ID can create a time-of-check/time-of-use gap. Include tenant/owner and version predicates in the mutation itself.

8. Observability and SLO signals

Define a "good" API request: an eligible request completed within 500 ms and returned the expected non-5xx outcome; exclude explicitly rejected abusive traffic according to the SLO policy. Track:

- request count, outcome, and duration by low-cardinality route template;
- admission rejection, deadline expiry, and retry attempts;
- idempotency first-use, replay, fingerprint conflict, and stuck age;
- inventory conflict and database transaction retry;
- connection pool wait and timeout;
- outbox oldest-unpublished age, pending count, attempts, and permanent failures;
- fulfillment consumer lag and deduplication rate;
- saturation: CPU, allocation, GC, request concurrency, pool occupancy.

Alert on user impact and backlog age, not individual expected validation failures. A trace should connect request, transaction metadata, outbox event ID, and relay publish without putting personal data into span attributes.

9. Test strategy

Unit tests

- domain rejects empty, duplicate-policy, excessive quantity, and too many lines;
- canonical fingerprint is stable for semantically equivalent request DTOs;
- ETag/version mapping and cursor codec reject malformed inputs;
- retry classifier never retries validation or authorization failures.

MVC/security tests

- missing/invalid token, wrong audience, forbidden ownership, validation body;
- exact status, media type, `Location`, ETag, and Problem Details shape;
- unknown fields and enum evolution follow the documented policy;
- maximum payload/header sizes are enforced at the relevant deployment layer.

Database integration tests

- two concurrent reservations cannot oversell;
- two concurrent identical keys create one order and outbox event;
- same key/different fingerprint is rejected;
- transaction rollback leaves no partial inventory/order/idempotency completion;
- keyset query uses correct tuple ordering and target-engine plan;
- version predicate returns update count zero on conflict.

End-to-end tests

- lost-response simulation followed by a replay returns the same order;
- relay crash after publish produces a duplicate that fulfillment deduplicates;
- graceful shutdown removes readiness and preserves committed work;
- management endpoints are inaccessible from the public security boundary.

TRY IT | 10. Interview question bank and model checkpoints

- 1 **Why not publish directly to Kafka inside `@Transactional`?**
Checkpoint: the database and broker are independent atomic domains. A Java method cannot make their commits atomic. Use an outbox or an explicitly selected distributed protocol; design duplicate delivery.
- 2 **Why store the idempotency result instead of only the key?**
Checkpoint: after a lost response, the server must reproduce or locate the original logical outcome. A bare key says "seen" but cannot answer what happened.
- 3 **How do you prevent overselling?**
Checkpoint: use a database-enforced atomic conditional update or appropriate locking/isolation within a short transaction; inspect update count and test concurrency on the target engine.
- 4 **What is the list endpoint's complexity?**
Checkpoint: with a matching composite index, keyset lookup plus `limit` can avoid work proportional to deep offset, but the real claim depends on the query plan, selectivity, row visibility, and fetches. State response serialization as `0(pageSize)`.
- 5 **Can an ETag be the ORM version number?**
Checkpoint: it can be encoded into a representation validator if the version changes whenever that representation changes and the API follows strong/weak validator rules. A partial projection or external computed fields may invalidate that assumption.
- 6 **What does `@WebMvcTest` prove here?**
Checkpoint: selected MVC mapping, conversion, validation, advice, and configured security behavior. It does not prove the transaction, SQL, outbox, or complete production context.
- 7 **How would you handle payment next?**
Checkpoint: create a persistent order/payment workflow with idempotent provider calls, deadlines, callback verification, compensating transitions, and reconciliation. Do not hold a database transaction over the network call.
- 8 **What breaks first during overload?**
Checkpoint: identify measured scarce resources-request concurrency, connection pool, database locks/CPU, relay backlog, broker/downstream capacity. Bound admission and shed work before queues consume the entire deadline.

SDE-2 design extensions

- split inventory into another service and design reservation expiration plus saga compensation;
- add multi-region reads and state the consistency/read-your-writes contract;
- support customer deletion while retaining legally required order records and minimizing personal data;
- migrate cursor format without invalidating every active cursor abruptly;
- run a zero-downtime schema change that makes `client_note` encrypted or separately stored.

TRY IT | 11. Exercises

- 1 Write the idempotency-record state diagram, including two callers, transaction rollback, lease expiry, and retention deletion.
- 2 Design SQL and application flow for reserving five SKUs without deadlock-prone arbitrary order.
- 3 Specify the outbox relay claim algorithm for multiple relay instances and explain duplicate windows.
- 4 Add cancellation of a pending order. Define allowed transitions, optimistic concurrency, event contracts, and replay behavior.
- 5 Produce a 30-minute interview answer: estimates, API, schema, transaction, failure table, security, observability, and scale evolution.

12. Final readiness checklist

- ☐ Requirements distinguish synchronous commit from asynchronous fulfillment.
- ☐ HTTP, validation, Problem Details, pagination, conditional update, and idempotency contracts are explicit.
- ☐ Security identity flows into tenant-scoped database predicates.
- ☐ One local transaction protects inventory, order, outbox, and idempotency outcome.
- ☐ Every cross-system boundary has a duplicate and timeout strategy.
- ☐ Queueing and scarce resources are bounded.
- ☐ Metrics, logs, traces, probes, and shutdown have operational semantics.
- ☐ Tests are matched to claims and include target-engine concurrency.

Primary references

- Spring Framework Reference, "Spring Web MVC": <https://docs.spring.io/spring-framework/reference/web/webmvc.html>
- Spring Framework Reference, "Transaction Management": <https://docs.spring.io/spring-framework/reference/data-access/transaction.html>
- Spring Security Reference: <https://docs.spring.io/spring-security/reference/>
- Spring Boot Reference, "Actuator": <https://docs.spring.io/spring-boot/reference/actuator/>
- RFC 9110, "HTTP Semantics": <https://www.rfc-editor.org/rfc/rfc9110>
- RFC 9111, "HTTP Caching": <https://www.rfc-editor.org/rfc/rfc9111>
- RFC 9457, "Problem Details for HTTP APIs": <https://www.rfc-editor.org/rfc/rfc9457>

Version boundary: this case uses Java 21 and Jakarta-era Spring concepts but intentionally avoids a particular Boot patch release. Dependency-specific annotations and defaults must be checked against the project's managed BOM. Later Java features are optional deltas, not assumed syntax or runtime behavior.

SDE-2 CORE CHAPTER

Chapter 5 - Executable Spring Boundary Model

This dependency-free Java 21 model makes idempotency, optimistic concurrency, error contracts, and bounded retry policy executable without pretending to reimplement Spring.

JAVA (continued 1/9)

```
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.time.ZoneOffset;
import java.util.ArrayList;
import java.util.Base64;
import java.util.Comparator;
import java.util.HashMap;
import java.util.HexFormat;
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.Optional;

/**
 * Dependency-free Java 21 models for API-boundary interview exercises.
 *
 * <p>This is not a distributed idempotency store. It makes the state and
 * invariants executable; production code must use durable atomic storage.
 */
public final class SpringBoundaryModel {
    private SpringBoundaryModel() {}

    public record ScopedKey(String tenant, String operation, String key) {
        public ScopedKey {
            requireText(tenant, "tenant");
            requireText(operation, "operation");
            requireText(key, "key");
            if (key.length() > 128) {
```

JAVA (continued 2/9)

```
        throw new IllegalArgumentException("key is too long");
    }
}

public record Request(String customerId, List<Line> lines, String note) {
    public Request {
        requireText(customerId, "customerId");
        lines = List.copyOf(Objects.requireNonNull(lines, "lines"));
        if (lines.isEmpty() || lines.size() > 100) {
            throw new IllegalArgumentException("line count must be 1..100");
        }
        note = note == null ? "" : note;
        if (note.length() > 200) {
            throw new IllegalArgumentException("note is too long");
        }
    }

    public String canonicalForm() {
        var sorted = new ArrayList<>(lines);
        sorted.sort(Comparator.comparing(Line::sku)
            .thenComparingInt(Line::quantity));
        var builder = new StringBuilder(customerId).append('|');
        for (Line line : sorted) {
            appendLengthPrefixed(builder, line.sku());
            builder.append(':').append(line.quantity()).append('|');
        }
        appendLengthPrefixed(builder, note);
        return builder.toString();
    }
}

public record Line(String sku, int quantity) {
```

JAVA (continued 3/9)

```
    public Line {
        requireText(sku, "sku");
        if (quantity < 1 || quantity > 10_000) {
            throw new IllegalArgumentException("quantity must be 1..10000");
        }
    }
}

public sealed interface ReservationResult
    permits Owner, Replay, InProgress, PayloadConflict {

    public record Owner(long attemptToken) implements ReservationResult {
    }

    public record Replay(String resourceId) implements ReservationResult {
        public Replay {
            requireText(resourceId, "resourceId");
        }
    }

    public record InProgress(Instant retryAfter) implements ReservationResult {
        public InProgress {
            Objects.requireNonNull(retryAfter, "retryAfter");
        }
    }

    public record PayloadConflict(String expectedFingerprint)
        implements ReservationResult {
    }

    private enum State { IN_PROGRESS, SUCCEEDED }
}
```

JAVA (continued 4/9)

```
private static final class Entry {
    private final String fingerprint;
    private final long attemptToken;
    private final Instant leaseUntil;
    private final Instant expiresAt;
    private State state = State.IN_PROGRESS;
    private String resourceId;

    private Entry(String fingerprint, long attemptToken,
        Instant leaseUntil, Instant expiresAt) {
        this.fingerprint = fingerprint;
        this.attemptToken = attemptToken;
        this.leaseUntil = leaseUntil;
        this.expiresAt = expiresAt;
    }
}

/** Thread-safe only within one process; used to exercise the contract. */
public static final class IdempotencyRegistry {
    private final Map<ScopedKey, Entry> entries = new HashMap<>();
    private final Clock clock;
    private final Duration lease;
    private final Duration retention;
    private long nextAttemptToken = 1;

    public IdempotencyRegistry(
        Clock clock, Duration lease, Duration retention) {
        this.clock = Objects.requireNonNull(clock, "clock");
        this.lease = requirePositive(lease, "lease");
        this.retention = requirePositive(retention, "retention");
        if (retention.compareTo(lease) < 0) {
            throw new IllegalArgumentException(
                "retention must not be shorter than lease");
        }
    }
}
```

JAVA (continued 5/9)

```
    }  
}  
  
public synchronized ReservationResult reserve(  
    ScopedKey key, String fingerprint) {  
    Objects.requireNonNull(key, "key");  
    requireText(fingerprint, "fingerprint");  
    Instant now = clock.instant();  
    Entry current = entries.get(key);  
    if (current != null && !current.expiresAt.isAfter(now)) {  
        entries.remove(key);  
        current = null;  
    }  
    if (current == null) {  
        long token = nextAttemptToken++;  
        entries.put(key, new Entry(fingerprint, token,  
            now.plus(lease), now.plus(retention)));  
        return new Owner(token);  
    }  
    if (!MessageDigest.isEqual(  
        current.fingerprint.getBytes(StandardCharsets.UTF_8),  
        fingerprint.getBytes(StandardCharsets.UTF_8))) {  
        return new PayloadConflict(current.fingerprint);  
    }  
    if (current.state == State.SUCCEEDED) {  
        return new Replay(current.resourceId);  
    }  
    return new InProgress(current.leaseUntil);  
}  
  
public synchronized void complete(  
    ScopedKey key, long attemptToken, String resourceId) {  
    Entry current = entries.get(Objects.requireNonNull(key, "key"));
```

JAVA (continued 6/9)

```
        if (current == null || current.attemptToken != attemptToken
            || current.state != State.IN_PROGRESS) {
            throw new IllegalStateException("attempt does not own reservation");
        }
        requireText(resourceId, "resourceId");
        current.resourceId = resourceId;
        current.state = State.SUCCEEDED;
    }

    public synchronized int size() {
        return entries.size();
    }
}

public record OrderView(String id, Instant createdAt) {
    public OrderView {
        requireText(id, "id");
        Objects.requireNonNull(createdAt, "createdAt");
    }
}

public record Cursor(Instant createdAt, String id) {
    public Cursor {
        Objects.requireNonNull(createdAt, "createdAt");
        requireText(id, "id");
    }
}

/**
 * Returns a descending keyset page. Input is sorted defensively so the
 * executable model does not hide the required total ordering.
 */
public static List<OrderView> pageAfter(
```

JAVA (continued 7/9)

```
        List<OrderView> source, Cursor after, int limit) {
    if (limit < 1 || limit > 100) {
        throw new IllegalArgumentException("limit must be 1..100");
    }
    Comparator<OrderView> newestFirst = Comparator
        .comparing(OrderView::createdAt).reversed()
        .thenComparing(OrderView::id, Comparator.reverseOrder());
    return source.stream()
        .sorted(newestFirst)
        .filter(order -> after == null
            || order.createdAt().isBefore(after.createdAt())
            || (order.createdAt().equals(after.createdAt())
                && order.id().compareTo(after.id()) < 0))
        .limit(limit)
        .toList();
}

public static String strongEtag(String resourceId, long version) {
    requireText(resourceId, "resourceId");
    if (version < 0) {
        throw new IllegalArgumentException("version must be nonnegative");
    }
    String token = resourceId + "-v" + version;
    String safe = Base64.getUrlEncoder().withoutPadding()
        .encodeToString(token.getBytes(StandardCharsets.UTF_8));
    return "'" + safe + "'";
}

public static String fingerprint(Request request) {
    try {
        byte[] digest = MessageDigest.getInstance("SHA-256")
            .digest(request.canonicalForm()
                .getBytes(StandardCharsets.UTF_8));
    }
```

JAVA (continued 8/9)

```

        return HexFormat.of().formatHex(digest);
    } catch (NoSuchAlgorithmException impossible) {
        throw new AssertionError("Java requires SHA-256", impossible);
    }
}

public static void main(String[] args) {
    Clock fixed = Clock.fixed(
        Instant.parse("2026-01-01T00:00:00Z"), ZoneOffset.UTC);
    var registry = new IdempotencyRegistry(
        fixed, Duration.ofSeconds(30), Duration.ofHours(24));
    var key = new ScopedKey("tenant-1", "create-order", "request-7");
    var request = new Request("customer-1",
        List.of(new Line("BOOK-21", 2)), "reception");
    String hash = fingerprint(request);

    ReservationResult first = registry.reserve(key, hash);
    assert first instanceof Owner;
    long token = ((Owner) first).attemptToken();

    assert registry.reserve(key, hash) instanceof InProgress;
    assert registry.reserve(key, fingerprint(new Request("customer-1",
        List.of(new Line("BOOK-21", 3)), "reception")))
        instanceof PayloadConflict;

    registry.complete(key, token, "order-91");
    ReservationResult replay = registry.reserve(key, hash);
    assert replay.equals(new Replay("order-91"));
    assert registry.size() == 1;

    Instant t = Instant.parse("2026-01-01T10:00:00Z");
    List<OrderView> orders = List.of(

```

JAVA (continued 9/9)

```
        new OrderView("0-3", t),
        new OrderView("0-2", t),
        new OrderView("0-1", t.minusSeconds(1)));
    assert pageAfter(orders, null, 2)
        .equals(List.of(orders.get(0), orders.get(1)));
    assert pageAfter(orders, new Cursor(t, "0-2"), 2)
        .equals(List.of(orders.get(2)));

    assert strongEtag("0-91", 8).startsWith("\");
    assert strongEtag("0-91", 8).endsWith("\");
    System.out.println("SpringBoundaryModel assertions passed");
}

private static void appendLengthPrefixed(
    StringBuilder builder, String value) {
    builder.append(value.length()).append(':').append(value);
}

private static void requireText(String value, String name) {
    if (value == null || value.isBlank()) {
        throw new IllegalArgumentException(name + " must not be blank");
    }
}

private static Duration requirePositive(Duration value, String name) {
    Objects.requireNonNull(value, name);
    if (value.isZero() || value.isNegative()) {
        throw new IllegalArgumentException(name + " must be positive");
    }
    return value;
}
```

Part II - Practice and Handoff

Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: container, bean, controller, and HTTP contracts
- Interview Core: REST errors, pagination, transactions, and tests
- SDE-2 Follow-up: idempotency, security, and overload
- Challenge: defend the worked order API end to end

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

[Previous volume: 18G - Advanced Java G: Question Bank, Study Plan, and Reference](#)

[Series index](#)

[Next volume: 18I - Advanced Java I: Persistence, SQL, JPA, and Caching](#)

TRY IT | Completion check

- ☐ I can recognize the core patterns without being told the data structure or algorithm.
- ☐ I can implement the representative Java templates from memory.
- ☐ I can explain why each implementation is correct.
- ☐ I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- ☐ I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Complete the learning steps in order when rebuilding from fundamentals, or enter at the first step whose readiness checks expose a gap. Stable volume numbers remain in filenames; the steps below show the recommended reading order. Click a title when your PDF viewer permits local sibling-file links.

STEP	FOCUSED LEARNING PATH
1	Java Foundations for Problem Solving
2	Time and Space Complexity
3	Number Systems and Math Foundations for DSA Interviews
4	Bit Manipulation in Java
5	Loop Mastery, Patterns, and Index Calculations
6	Arrays and Array Problem-Solving Patterns
7	Strings and String Problem-Solving Patterns
8	Hashing: Maps, Sets, Frequency, and Prefix State
9	Recursion and Backtracking in Java
10	Linked Lists: Pointer Reasoning and Mutation
11	Stacks, Queues, Deques, and Monotonic Patterns
12	Binary Search: Bounds, Answers, and Invariants
13	Trees, Binary Search Trees, and Tries
14	Heaps, Priority Queues, Selection, and Top-K
15	Graphs: Traversal, Ordering, Paths, and Union-Find
16	Greedy Algorithms: Recognition, Proofs, and Scheduling
17	Dynamic Programming: State, Transitions, and Optimization
18	Advanced Java Engineering and the SDE-2 Interview (Parts A-J)

Learning Step 3 uses a linked Number Systems foundations volume and workbook; the final advanced stage uses a ten-part collection, including a three-volume backend specialist track. These splits keep every file focused and printable.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: [LinkedIn](#) | [GitHub](#)

Copyright and Publishing Notes

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and PDFs are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: <https://github.com/vinayreddykalluri/SDE2-Interview-Handbook>. Attribution does not imply endorsement. Java and related marks are owned by their respective holders.

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Volume 18 of 18 - Part H of J. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.